

DOCUMENTAȚIE PROIECT: SISTEM ȘAH

Arhitectură Full-Stack .NET 10 & React 19

1. Arhitectură și Decizii Tehnice

Sistemul este organizat într-o structură **decuplată** pentru a asigura o experiență de utilizare fluidă (latență minimă pe frontend) și o autoritate centrală pentru gestionarea timpului (backend).

- Chess.Api:** ASP.NET Core Minimal API - gestionează sesiunea și ceasul.
- Chess.Core:** Modele de domeniu partajate între straturile backend-ului.
- Frontend:** React 19, TypeScript, Vite, Tailwind CSS 4 și Framer Motion.

2. API Backend și Endpoint-uri

Base URL: `http://localhost:5138`

Metodă	Endpoint	Descriere și Funcționalitate
GET	<code>/api/board</code>	Returnează lista inițială de piese (ID, tip, culoare, coordonate).
GET	<code>/api/game-clock</code>	Sincronizează secunde rămase pentru Alb/Negru și tura curentă.
POST	<code>/api/game-clock/turn</code>	Body: <code>{"turn": "White" "Black"}</code> . Comută cronometrul activ.
POST	<code>/api/game-clock/reset</code>	Resetează timpii la 10:00 și tura la White.

3. Ghid de Rulare și Flux Runtime

3.1. Pornirea sistemului

```
# Backend
cd Chess.Api
dotnet run

# Frontend
cd frontend
npm install
npm run dev
```

3.2. Fluxul de execuție

1. Frontend-ul inițializează tabla prin GET /api/board.
2. Un efect de tip setInterval (1s) apelează /api/game-clock pentru a menține timer-ul sincronizat.
3. La fiecare mutare validă, frontend-ul execută POST /api/game-clock/turn pentru a opri scurgerea timpului jucătorului curent.

4. Moduri de Joc și UX

1 vs 1 (Local): Jucătorii mută alternativ pe același ecran. Interfața blochează selecția pieselor care nu aparțin culorii aflate la rând.

AI Mode: După mutarea Albului, sistemul activează getBestMoveFromAI. Acesta analizează BoardState, alege o mutare validă și o execută cu un delay vizual (simulate thinking).

Feedback Vizual:

- **Animații:** Utilizarea Framer Motion pentru tranziții line ale pieselor.
- **Status:** Indicatori textuali pentru "AI is thinking" sau "Your Turn".
- **Highlighting:** Markere vizuale pentru celulele unde piesa selectată poate muta legal.

5. Securitate și Limitări Curente

Problemă de Securitate: Validarea mutărilor se face exclusiv în chessRules.ts (Frontend). Un atacator poate manipula API-ul pentru a muta piese ilegal sau a reseta ceasul.

- **Reguli lipsă:** Nu sunt implementate rocada, en-passant și promovarea pionului.
- **Checkmate:** Detectarea șah-matului este parțială; nu oprește forțat fluxul API.

6. Criterii de Acceptanță (MVP)

- Sistemul permite o partidă cap-la-cap fără erori de crash.
- Ceasul server-side este autoritatea finală pentru timp.
- Piesele respectă constrângerile de mișcare de bază (ex: Nebunul doar pe diagonală).

7. Implementări și Extensii Viitoare

7.1. Securitate și Validare Double-Check

Migrarea logicii de validare și în backend (C#) pentru a verifica fiecare mutare primită prin API. Orice mutare ilegală va rezulta într-un cod `400 Bad Request`, prevenind trișarea.

7.2. Persistență și Multiplayer Real-time

- **Bază de Date:** Salvarea istoricului mutărilor în SQL Server/PostgreSQL pentru a permite reluarea partidei.
- **SignalR:** Implementarea WebSockets pentru a permite jocul între doi utilizatori pe dispozitive diferite în timp real.

7.3. Engine AI Avansat

Înlocuirea logicii locale de AI cu un serviciu de micro-serviciu dedicat care rulează **Stockfish** sau integrarea completă a modelului Hugging Face pentru predicții bazate pe baze de date de mari dimensiuni (Grandmaster games).

7.4. UI/UX Pro

Implementarea unui sistem de "Replay" (istoricul mutărilor în notație algebrică) și a unui mod de analiză post-joc.